



ACITM

Year 2000 Toolkit

Copyright 1999 by ACI Worldwide Inc.

All rights reserved. All information contained in this document is confidential and proprietary to ACI Worldwide Inc., and has been delivered for use pursuant to license. No part of this document may be photocopied, electronically transferred, modified, or reproduced in any manner that is contrary to license without the prior written consent of ACI Worldwide Inc.

ACI, BASE24, BASE24-atm, BASE24-card, BASE24-from host maintenance, BASE24-mail, BASE24-pos, BASE24-teller, and InfoBase are registered trademarks of ACI Worldwide Inc.

BASE24-billpay, BASE24-InfoBase, BASE24 Remote Banking, BASE24-self service banking, BASE24-telebanking, NET24 Performance Monitoring subsystem, and NET24-XPNET are trademarks and products of ACI Worldwide Inc.

All other brand or product names are trademarks or registered trademarks of their respective companies or organizations.

Version Record

5/99, Version 1.05	Copyright and ACI Worldwide Inc. updated
6/98, Version 1.04	Added Native USERTCP/TCPLIBB and minor modifications
11/97, Version 1.03	Minor modifications and clarifications
10/97, Version 1.02	Minor modifications and clarifications
5/97, Version 1.01	Initial Printing

Table of Contents

Preface.....	V
Manual Summary	V
Prerequisites.....	V
Section 1 - Introduction.....	1-1
Overview of the Tools	1-1
Assumptions	1-2
Requirements	1-2
Warning!!	1-2
Section 2 - The Tools Available.....	2-1
SEEK	2-1
Year 2000 Library	2-2
TIMUPDT	2-2
MYTIME	2-3
Keyword List	2-3
Read Me	2-3
Build TCPLIBB in Native Mode.....	2-3
Section 3 - Analyzing The Source Files	3-1
SEEK	3-1
Section 4 - Creating Useful Network Components	4-1
SKELR2K	4-1
XNLIB2K (or D20LIB2K or D30LIB2K).....	4-1
NETWORK	4-2
Generic Simulator	4-2
TCPLIBB in Non-Native Mode	4-2
TCPLIBB in Native Mode	4-3
COBOL85 Compiler and C8LIB	4-3
Section 5 - Setting Up The Test Environment	5-1
Network Setup	5-1
Pathway Setup.....	5-2
COBOL85 Application Setup	5-2
Miscellaneous Setup.....	5-3
Section 6 - Running the Test Environment.....	6-1
Altering the Date and Time	6-1
Network Startup	6-2
Changing the Date and Time While Processes Are Running.....	6-3
Section 7 - Test Guidelines.....	7-1
Appendix A - Keywords used with SEEK.....	A-1

Appendix B - Tandem and ACI Date and Time Representations	B-1
Definitions	B-1
Date and Time Representations	B-1
GREGORIAN ARRAY	B-1
48-BIT TIMESTAMP	B-2
64-BIT JULIAN TIMESTAMP	B-2
ASCII CALENDAR DATES.....	B-2
BINARY CALENDAR DATE	B-3
LEAP YEAR PROCESSING.....	B-3
Tandem Date and Time System Procedures	B-3
ACI Date and Time Utilities.....	B-4
Appendix C - Standard Solutions to Common Year 2000 Problems	C-1
Determination of Century	C-1
Comparing YYMMDD Dates	C-2
Conversion of a Four-Digit Year to a Two-Digit Year	C-5
Conversion of a Two-Digit Year to a Four-Digit Year	C-6
Application File Names in YYMMDD Format	C-6
Date Fields Within Primary or Alternate Keys	C-7
Card Expiration Dates.....	C-8

Preface

Manual Summary

This manual will explain the purpose, functioning and setup of a set of tools developed by ACI. The set of tools discussed in this manual was developed specifically to facilitate testing the Year 2000 compatibility of BASE24 software.

This manual will describe each of the tools in the toolkit and give information on how to use each tool. Information is also included on how to setup and run a BASE24 network using the tools that will alter the network's perceived time.

Prerequisites

- A working knowledge of BASE24 is required to use the network tools provided.
- A working knowledge of Tandem commands and file structures is required to run SEEK.

ACI Worldwide Inc.

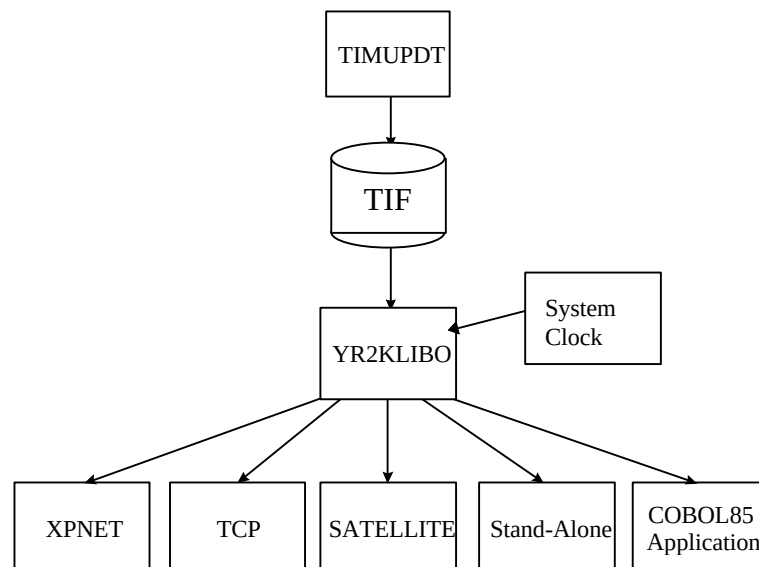
Section 1 - Introduction

ACI has developed a set of test tools and libraries to facilitate testing the Year 2000 compatibility of BASE24 software. The purpose of the testing is to ensure BASE24 will perform correctly on several dates that are related to the Year 2000. The libraries in this toolkit allow a developer to alter a process' perception of the current date and time. This will aid in the detection and isolation of problems related to generating timestamps, converting timestamps, comparing dates, and setting dates and times. The tools can be used to test programs written in TAL, COBOL85 or SCOBOL.

Overview of the Tools

There are two main types of tools included in this group of tools. The first tool is SEEK. It is an analysis tool that evaluates existing software and looks for text strings within the source code. SEEK is used to look for time and date related references within the software. Nothing in the software is changed as a result of the SEEK run. Each reference found by SEEK must be evaluated. Changes to the software are made using existing methodologies and change management.

The other tools defined in this manual are testing tools. They are used together to manipulate the perception of time by a BASE24 network or process while it is running. These tools can be used to change the time as seen by the BASE24 network or process without changing the source code and without modifying the system clock.



Assumptions

- The file name listed with each tool is where the tool is located on the tape provided.
- The tools sent on the tape will be loaded on \$volume.BA10YOBJ.
- Network components to use the time manipulation tools will be created on \$volume.YR2KTEST.
- A COBOL compiler that uses the time manipulation tools will be created on \$volume.YR2KCOB.

Requirements

- SEEK requires at least a Guardian "D" operating system.
- The network tools (XPNET and SPANNET) must be shipped after May 1, 1997 to work with the Year 2000 tools. The BASE file found on a customer's SPANNET subvolume is the object file that contains the changes for Year 2000 testing.

The necessary release procedures are the following:

XPNET 3.0	All versions
XPNET 2.1	REL2^VER1^EXEC04^970109
XPNET 2.0	REL2^VER0^EXEC12^970120
SPANNET 3.7	REL3^EXEC7E^08JAN97

- The Generic Simulator must be shipped after May 1, 1997 to work with the Year 2000 tools.

Warning!!

These tools are provided to help determine how BASE24 will respond on a particular date (past or future). Use of the tools does not guarantee that a BASE24 network or product will perform correctly when a particular date actually occurs. There is no warranty, express or implied, with the distribution or use of the tools.

When using SEEK to scan source code, the output is only as good as the list of target strings (keywords). SEEK will find only those things for which it is directed to search. The user of the tool must carefully evaluate the programs being scanned to be sure the input list of target strings is complete.

Section 2 - The Tools Available

The tool SEEK has been developed to search for multiple keywords in one pass through a file or set of files. This tool will allow code to be scanned for date and time processing variables.

ACI has also developed a set of tools (YR2KLBO and TIMUPDT) that can be used to modify the apparent clock time within a process. Any Tandem object can be bound together with YR2KLBO. These library procedures then intercept calls to various time utilities so the process being tested perceives a clock time that is different from the actual system clock.

SEEK

FILE - BA10YOBJ.SEEK

DESCRIPTION - SEEK is a generic search tool that can be used to search source files for various keywords (target strings). For our purposes, keywords are the field names, variable names and procedure names that typically occur when processing dates and times in BASE24 modules; e.g. "year", "yr" and "yy."

It optionally scans a single file, subvolume, volume or system, or multiple files, subvolumes, volumes, or systems. It can search for multiple keywords in one pass through a file or set of files. SEEK outputs the entire line of code that contains a keyword along with the line number. Output can be directed to the screen, spooler or an edit file. It optionally reads input from an edit file. SEEK is not redundant. That is, if two keywords appear on the same line of code, that line of code is not output twice. It optionally suppresses the output of a line if it is a comment. It outputs the name of every file scanned, or outputs the names of only those files that contain a keyword. It can scan a single file type or multiple file types (ACIAID, C, COBOL, DDL, EMS, FIX, MAKE, SCOBOL, SQLCI, TACL, TAL, TEMPLT, OTHER or ALL). Seek requires customers to be on a "D" operating system.

The results of the SEEK file scan are only as good as the list of target strings it receives. SEEK will find only those things for which it is directed to search. The user of the tool must carefully evaluate the programs being scanned to be sure the input list of target strings is complete.

Year 2000 Library

FILE - BA10YOBJ.YR2KLIBO non-Native mode
 BA10YOBJ.YR2KLIBN Native mode

DESCRIPTION - The Year 2000 Library contains replacement procedures for the three Guardian procedures that generate timestamps: JULIANTIMESTAMP, TIME and TIMESTAMP. This library can be used to test all BASE24 modules. It can be:

- Bound with SKEL and used as a library for XPNET application processes.
- Bound with XPNET itself.
- Bound with any other TAL object file.
- Bound with USERTCP and TCPLIBB for testing SCOBOL applications.
- Incorporated in COBOL85 object files.
- Specified as the LIB parameter in a TACL RUN command.
- Included in a TAL ?SEARCH compiler directive.

The library procedures invoke the actual Guardian procedures. Before returning a timestamp to the caller, however, the library procedures add a "time increment value" to the timestamp. In this manner, a process thinks it is processing on any date the user chooses.

The Year 2000 libraries allow a programming team to roll the date forward and backward without requiring dedicated hardware. Without the Year 2000 tools, a programming team would have to be synchronized across all testers whenever the clock was changed. The tools also eliminate interruptions to other users on the testing or development Tandem node.

TIMUPDT

FILE - BA10YOBJ.TIMUPDT

DESCRIPTION - TIMUPDT is a simple stand-alone program run from TACL. It accepts a date and/or time from the user, calculates the difference between that date/time and the current date/time (the increment value), and writes it to the Time Increment File (TIF) file which is read by the Year 2000 Library procedures. Whenever the Year 2000 Library procedures are invoked, they automatically detect if this file has been modified and obtain the new increment value.

TIMUPDT must be run at least once before testing begins to create a TIF, even if no change in time is desired. Otherwise, the process using the Year 2000 Libraries willabend with no apparent cause.

MYTIME

FILE - BA10YOBJ.MYTIME

DESCRIPTION - MYTIME is a simple stand-alone program run from TACL. It returns to the terminal the current date and time as perceived by the processes being tested. In other words, it returns the current system date/time + the time increment value in the TIF.

Keyword List

FILE - BA10YOBJ.KEYWORD

DESCRIPTION - A list of keywords that can be used as a starting point to create an input file for SEEK to search for time and date related target strings. Instructions on how to use the file are included within the file.

Read Me

FILE - BA10YOBJ.AAREADME

DESCRIPTION - A description of the files on the BA10YOBJ subvolume and the installation steps for creating the TCPLIBB in Native mode.

Build TCPLIBB in Native Mode

FILE - BA10YOBJ.YBLDTCPN

DESCRIPTION - Input information for the Native Link Editor (NLD) to build the TCPLIBB for systems D40 and later (native). This example can be used in place of BA10UTCP.BLDTCPN to incorporate the native YR2K library into TCPLIBB.

ACI Worldwide Inc.

Section 3 - Analyzing The Source Files

The tool SEEK is provided with the Year 2000 testing tools. The purpose of SEEK is to search source code for target strings (keywords). It allows multiple strings to be searched for simultaneously. Strings to search for are put in an input file. The results of the search are only as good as the input list provided. A sample list is provided in Appendix A. This list is an example of the types of variable names to be considered in setting up the search list. The input list may require customization to suit a customer's environment. Results of the search are written to an output file.

SEEK requires customers to be on at least a "D" operating system.

SEEK

SEEK / IN <INFILE>, OUT <OUTFILE> / <TARGET STRINGS>,
<FLAGS>,<FILENAME PATTERNS>, <FILE TYPES>

Recommended default overrides are flags -c, +q, and a list of specific file types. Typing SEEK with no parameters will display helpful information.

<INFILE> - Optional. Specifies the name of the infile. The infile is an edit file that can contain any of the startup parameters in any order separated by commas. All startup parameters must be in the command string or all startup parameters must be in the infile. They are mutually exclusive. A sample infile is found on BA10YOBJ.KEYWORD.

<OUTFILE> - Optional. Specifies where search results are displayed. The outfile may be a terminal, spooler location or edit file. If a nonexistent edit file is specified, one will be created. If the file exists, results will be appended to the end of the file. DEFAULT is the home terminal.

<TARGET STRINGS> - Required. Specifies the character strings for which to search. Each target string must be enclosed by double-quotes, single-quotes or parentheses. Multiple target strings must be comma-separated.

<FLAGS> - Optional. Specifies search and display aspects. Flags must be preceded by a plus or minus sign. Multiple flags must be comma-separated.

B (BOTH) Controls the case sensitivity of target strings. +B = case insensitive (DEFAULT). -B = case sensitive.

C (COMMENTS) Controls the display of commented lines in the source file types listed below (except OTHER). +C = display a line if it contains a target string within a comment (DEFAULT). -C = do not display a line if it contains a target string within a comment.

E (EVERY) Controls match criteria when searching for multiple target strings. +E = display a line if it contains every target string. -E = display a line if it contains any target string (DEFAULT).

Q (QUIET) Controls the display of file names. +Q = display the name of a file only if it contains a target string. -Q = display the name of every file searched (DEFAULT).

<FILENAME PATTERNS> Optional. Specifies the set of files to search. Wildcard characters are permitted. Multiple filename patterns must be comma-separated. DEFAULT is \$<CURRENT VOL>.<CURRENT SUBVOL>.*.

<FILE TYPES> Optional. Specifies the types of files to search. Multiple file types must be comma-separated.

- **ACIAID** ACI-proprietary ACIAID source files
- **C** ACI-proprietary C source files
- **COBOL** ACI-proprietary COBOL source files
- **DDL** ACI-proprietary DDL source files
- **EMS** ACI-proprietary EMS source files
- **FIX** ACI-proprietary FIX source files
- **MAKE** ACI-proprietary MAKE files
- **SCOBOL** ACI-proprietary SCOBOL source files
- **SQLCI** ACI-proprietary SQLCI source files
- **TACL** ACI-proprietary TACL source files
- **TAL** ACI-proprietary TAL source files
- **TEMPLT** ACI-proprietary TEMPLT source files
- **OTHER** files types other than the above or files of unrecognized type
- **ALL** all file types (DEFAULT)

An ACI-proprietary file is a file that has been cataloged by ACI and has a Software Management fix maintenance line at the beginning of the file. The information in this line is used by SEEK to determine the language in which the file is written. If SEEK cannot find a fix maintenance line or does not recognize the language indicator within the fix maintenance line, the file is treated as "OTHER", or unrecognized type. A file of unrecognized type can be searched. Comment suppression is not possible because SEEK cannot determine the language in which the file is written.

For example:

- If file types DDL and TAL are specified, SEEK will search only ACI-proprietary DDL and TAL source files. Files of all other types will be skipped.
- If file types DDL and OTHER are specified, SEEK will search only ACI-proprietary DDL source files and files of unrecognized type. Files of all other types will be skipped. If the "-c" flag is specified, comments will be suppressed in the DDL source files but not in the files of unrecognized type.
- If file type ALL is specified, SEEK will search all files regardless of type. If the "-c" flag is specified, comments will be suppressed in the files of recognized type but not in the files of unrecognized type.

ACI Worldwide Inc.

Section 4 - Creating Useful Network Components

In order to make the best use of the Year 2000 Library when running a test network, it will be useful to create some of the following components. The Year 2000 Library referred to in this section is the object file YR2KLIBO described in the section on available tools. A subvolume called YR2KTEST can be used to store most of the components created. A separate subvolume, YR2KCOB, is needed for the COBOL components.

The following files are NOT included with the Year 2000 Toolkit, but must be created by the user for each test site. **The bind information given should be used as an example only.** The bind commands must be tailored for each system depending on which version of the network and which operating system are present. The bind commands can be entered interactively or using an OBEY file.

SKELYR2K

FILE - YR2KTEST.SKELYR2K

DESCRIPTION - This is the network library (SKELB) bound with the Year 2000 Library. A typical use of this file would be to specify it as the library of an XPNET application process (Settlement, Authorization, Device Handlers, etc.) in the NEF or online via the NCS screen or NCPCOM. Use the current SKELB as the starting point for this bind. If that SKELB contains the migration library, be sure the new SKELB also contains that library. (The migration library SPANNET.MIGNLIB is used for high PIN processing. Look in the appropriate XPNET Implementation Guide for more information.)

BIND INFORMATION -

```
add * from $volume.SPANNET.SKELB
add * from $volume.BA10YOBJ.YR2KLIBO, delete
build SKELYR2K !, list * off
```

XNLIB2K (or D20LIB2K or D30LIB2K)

FILE - YR2KTEST.XNLIB2K (or D20LIB2K or D30LIB2K)

DESCRIPTION - This is the ITS network library (XNLIB or D20LIB or D30LIB) bound with the Year 2000 Library. A typical use of this file would be to specify it as the library of an ITS application process (EOPP, IEP, IAS, etc.) in the NEF or online via the NCS screen or NCPCOM.

BIND INFORMATION (for XNLIB2K) -

add * from \$volume.XNLIB.XNLIB
add * from \$volume.BA10YOBJ.YR2KLBO, delete
build XNLIB2K !, list * off

NETWORK

FILE - YR2KTEST.NETWORK

DESCRIPTION - This is the XPNET object bound with the Year 2000 Library. You must run this object when testing application processes. If not, XPNET's perception of time will differ from that of the applications, with unpredictable results.

The Network must have been shipped from ACI after May 1, 1997.

BIND INFORMATION -

add * from \$volume.SPANNET.NETWORK
add * from \$volume.BA10YOBJ.YR2KLBO, delete
build NETWORK !, list * off

Generic Simulator

FILE - YR2KTEST.GSIMYR2K

DESCRIPTION - This is the Generic Simulator object bound with the Year 2000 Library. The Generic Simulator must have been shipped from ACI after May 1, 1997.

BIND INFORMATION -

add * from \$volume.subvolume.GSIMV205 (or later)
add * from \$volume.BA10YOBJ.YR2KLBO, delete
build GSIMYR2K !, list * off

TCPLIBB in Non-Native Mode

FILE - YR2KTEST.TCPLIBB

DESCRIPTION - This is the Pathway TCPLIBB bound with the Year 2000 Library non-native mode. Configure this file in your Pathway environment to test SCOBOL applications if your system runs in non-native mode.

It is necessary to duplicate USERTCP on YR2KTEST to avoid library conflicts.

BIND INFORMATION -

add * from \$volume.subvolume.TCPLIBB
add * from \$volume.BA10YOBJ.YR2KLIBO, delete
build TCPLIBB!, list * off

TCPLIBB in Native Mode

FILE - YR2KTEST.TCPLIBB

DESCRIPTION - This is the Pathway TCPLIBB, linked with the Year 2000 Library in native mode. Configure this file in your Pathway environment to test SCOBOL applications if your system runs in native mode.

It is necessary to duplicate USERTCP on YR2KTEST to avoid library conflicts.

NLD INFORMATION - A sample file to build TCPLIBB and include the Year 2000 Library in native mode is BA10YOBJ.YBLDTCPN. This file adds the native mode Year 2000 Library, BA10YOBJ.YR2KLIBN.

COBOL85 Compiler and C8LIB

LOCATION - YR2KCOB.*

DESCRIPTION - The YR2KCOB subvolume must contain the following files:

- C8LIB The COBOL85 run-time object library file
- COBOL85 The COBOL85 compiler
- COBOEXT The COBOL external declarations file
- BINSERV The Binder server
- SYMSERV The Symbol Table server

C8LIB (the COBOL85 run-time object library file) is bound with the Year 2000 Library, enabling the CURRENT-DATE function and ACCEPT verb to invoke the Year 2000 Library and return "altered" dates and times. The other files are duplicates of the same files usually found on \$SYSTEM.SYSTEM. They have not been altered in any way. Because the compiler always looks for C8LIB on its own subvolume, it is necessary to duplicate these files in the same subvolume as the modified C8LIB.

COBOL85 applications must be recompiled with this compiler to take advantage of the Year 2000 Library.

BIND INFORMATION -

add * from \$volume.subvolume.C8LIB
add * from \$volume.BA10YOBJ.YR2KLIBO, delete
build C8LIB !, list * off

ACI Worldwide Inc.

Section 5 - Setting Up The Test Environment

Several steps are needed to get the BASE24 Network and Pathway systems to point to the components created using the Year 2000 Libraries. Any COBOL applications must be recompiled using the compiler which will include the Year 2000 Libraries. Changes given are for an XPNET 2.0/2.1 Network. Similar changes are needed for an XPNET 3.0 Network.

Network Setup

1. Choose a Logical Network on which to test. Because Year 2000 testing involves altering the network's perception of the current time, it is best to appropriate a network for your exclusive use.

2. Change the network's "GO" file to run:

```
\<system>.$<volume>.YR2KTEST.NETWORK
```

This is the XPNET object bound with the Year 2000 Library.

3. For each network application process you intend to test, change the library file designation to:

```
\<system>.$<volume>.YR2KTEST.SKELYR2K
```

This is the network library (SKELB) bound with the Year 2000 Library. This change may be made in the NEFS or online via the NCS/NCPCOM.

If this is an ITS process, the library file should be:

```
\<system>.$<volume>.YR2KTEST.D30LIB2K
```

4. For each Generic Simulator process you intend to use, change the object file designation to:

```
\<system>.$<volume>.YR2KTEST.GSIMYR2K
```

This is the Generic Simulator object bound with SKELB and the Year 2000 Library. This change may be made in the NEFS or online via the NCS/NCPCOM.

Pathway Setup

Follow these steps to integrate the Year 2000 Library in a Pathway environment.

1. From PATHCOM, stop the TCP:
=STOP <tcp name>
2. Change the TCP's GUARDIAN-LIB and PROGRAM designations:
=ALTER <tcp name>, GUARDIAN-LIB
 \<system>.\$<volume>.YR2KTEST.TCPLIBB

=ALTER <tcp name>, PROGRAM
 \<system>.\$<volume>.YR2KTEST.USERTCP
3. Start the TCP:
=START <tcp name>

Anytime a SCOBOL application calls ACCEPT <accept-name> FROM DATE, ACCEPT <accept-name> FROM DAY, ACCEPT <accept-name> FROM DAY-OF-WEEK or ACCEPT <accept-name> FROM TIME, the Year 2000 Library will be invoked.

COBOL85 Application Setup

Follow these steps to include the Year 2000 Library in a COBOL85 application.

1. Change the application's make file or compile file to point to the COBOL85 compiler on the YR2KCOB subvolume.
 - a) If the application is compiled with MAKE, change the COBOL define from its standard value by adding the following to the application's make file:

COBOL = \<system>.\$<volume>.YR2KCOB.COBOL85
 - b) If the application is compiled without MAKE, change the COBOL85 command in the compile file to start

\<system>.\$<volume>.YR2KCOB.COBOL85.

2. Compile the application.

Anytime a COBOL85 application calls CURRENT-DATE, ACCEPT <accept-name> FROM DATE, ACCEPT <accept-name> FROM DAY, ACCEPT <accept-name> FROM DAY-OF-WEEK or ACCEPT <accept-name> FROM TIME, or ENTERs the TAL procedures TIME, TIMESTAMP or JULIANTIMESTAMP, the Year 2000 Library will be invoked. Recomilation is necessary to embed the Year 2000 Library in the application's object code.

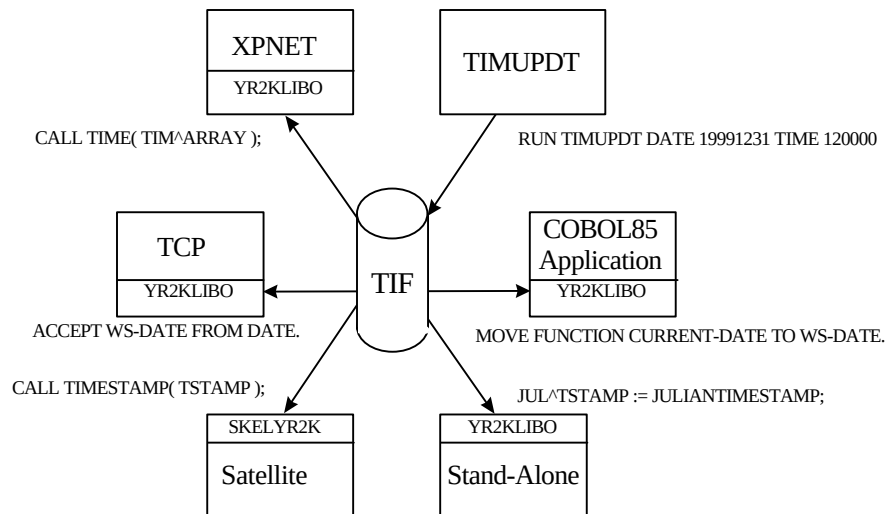
Miscellaneous Setup

1. If you are testing a stand-alone TAL process, the Year 2000 Library may be implemented in a couple of ways. Using BIND, you may bind \$volume.BA10YOBJ.YR2KLBO with the object file of the module you wish to test. You may also specify \$volume.BA10YOBJ.YR2KLBO as the LIB parameter in a RUN command from the TACL prompt.

ACI Worldwide Inc.

Section 6 - Running the Test Environment

Once a module is bound with the Year 2000 Library, its perception of the current date and time can be altered through the use of TIMUPDT. The relationship between TIMUPDT, the Time Increment File (TIF) and processes that use the Year 2000 Library is illustrated below.



Altering the Date and Time

There is one TIF for a Tandem user ID, located on the user's logon default volume/subvolume. The TIF for the Tandem user ID that starts the network is the only TIF used by TIMUPDT for that network. TIMUPDT accepts a date and/or time from the user and calculates the difference between that date/time and the current system date/time. This difference is the time increment value, which TIMUPDT subsequently writes to the TIF. TIMUPDT's command syntax is as follows:

```
run timupdt / out $<vol>.<subvol>.tif / date <yyyymmdd> time <hhmmss>
```

If no /out/ file is provided, TIMUPDT assumes the TIF is on the user's default volume/subvolume (this is recommended). If no TIF exists, TIMUPDT automatically creates one. If no date is provided, TIMUPDT defaults to the current system date. If no time is provided, TIMUPDT defaults to the current system time. TIMUPDT must be run at least once before testing begins to create a TIF, even if no change in time is desired. (Be sure a default volume and subvolume are defined for the user so a TIF can be created.) Otherwise, the process using the Year 2000 Libraries will abend with no apparent cause.

Whenever a process gets the current date and time with a call to the Guardian procedures JULIANTIMESTAMP, TIME, TIMESTAMP, ACCEPT or CURRENT-DATE, it actually calls one of the replacement procedures in the Year 2000 Library. If the TIF is not already open, the library procedures open it, expecting to find it on the user's default volume/subvolume. The library procedures check to see if the TIF has been updated and, if so, read the new time increment value from the TIF. The library procedures invoke the actual Guardian procedures and then add the time increment value to the timestamp, which is subsequently returned to the caller. In this manner, the process' perception of the current date and time is altered.

Network Startup

These steps are written for an XPNET 2.0/2.1 Network. Similar steps must be followed for an XPNET 3.0 Network.

1. Run TIMUPDT to create a TIF.
2. Start XPNET by obeying the network "GO" file. If you attempt to start XPNET and a TIF does not exist, the following message is displayed:

"Warning - \$<XPNET PPD> replied error 201 to the OPEN message - continuing"

If this occurs, use TIMUPDT to create a TIF. Restart XPNET.
3. From the NCS screen or NCPCOM, send the command "START SYSTEM (or NODE)<XPNET node name>".
4. Start the Generic Simulator, if needed. If you attempt to start the Generic Simulator outside the licensed interval, the following message is logged:

"GENSIM HAS NOT BEEN PROPERLY LICENSED"

If you see this message, contact your account representative to re-license the Generic Simulator.
5. Use TIMUPDT to set the date/time to the date/time you wish to test. Follow the rules in the section on Changing the Date and Time While Processes Are Running to advance to the desired date/time.
6. From the NCS screen or NCPCOM, start the application processes to be tested.

Changing the Date and Time While Processes Are Running

You may wish to change the date/time while a process is running. Depending on the process you are testing, you may be able to do so. Other processes may have to be stopped. Following are some guidelines to follow when the date/time is changed by TIMUPDT.

1. XPNET can remain running.
2. The Generic Simulator can remain running.
3. Pathway SCOBOL requesters and the TCP can remain running.
4. Pathway servers, typically COBOL85 applications, can remain running.
5. Application processes, such as Settlement, Authorization, Device Handlers, etc., must be handled with care. Some processes, such as POS Settlement, get a "startup time" during initialization. If the date/time is changed, a warmboot will not prompt a change in the startup time; the process has to be reinitialized. Also, a process' management of timers must be considered. A warmboot might not allow some timers to be properly altered if the date/time is changed. In general, application processes should be stopped and restarted when the date/time is changed.

ACI Worldwide Inc.

Section 7 - Test Guidelines

Thorough testing will be required to verify the functioning of BASE24 on various dates. These guidelines are just a starting point for a complete test plan.

1. The following dates are suggested dates for testing. Other dates should be tested as well:

January 1, 2000

February 29, 2000

January 1, 2001

February 28, 2001

January 1, 2004

February 29, 2004

2. Prior to testing a module, look up that module in SEEK's output file and visually analyze date/time processing. Careful analysis is crucial - it is likely that most problems will be found visually, prior to testing. Isolate potential problems and then test them.
3. Test each problematic date for a three day window when appropriate. e.g. December 31, 1999 through January 2, 2000.
4. Some things to test: (this list is by no means complete)
 - a) A transaction is in-flight while the clock hits a problematic date.
 - b) A transaction times out seconds after the clock hits a problematic date. Is the timeout handled properly?
 - c) Settlement and cutover occur while the clock hits a problematic date.
 - d) Transaction Log File retention period
 - e) Settlement reports retention period
 - f) Transaction Log File screen sort by date/time
 - g) Refresh impacting

- h) Terminals in time zones other than that of BASE24
 - i) ATM Inactivity Timing and Peak Load Timing
- 5. Also conduct any regression tests normally used when modifying the code.
- 6. If a fix is necessary, check Appendix A for Standard Solutions to Common Year 2000 Problems. There may be a standard fix that fits the problem.

Appendix A - Keywords used with SEEK

Following is a master list of keywords which was used during the analysis phase of the Year 2000 project for BASE24. Every effort has been made to create a list that is comprehensive. The many variations of "dat", "date", "tim" "ts" and "time" were derived to effectively filter unwanted matches on words such as "data", "update", "timer" and "timeout". These four words alone occur thousands of times in BASE24 source code. Note: represents one blank character.

Keywords

dat

dat^

dat;

dat,

dat.

dat-

dat_

dat)

dat(

dat[

dat"

dat:

date

^date

-date

.date

"date

@date

_date

(date

*date

,date

'date

Keywords

tim

tim^

tim;

tim,

tim.

tim-

tim_

tim)

tim(

tim[

tim"

tim:

tim1

tim2

tim3

time

time^

time;

time,

time.

time-

time_

time)

time(

time[

time"

time:

time1

time2

time3

Keywords

ts

ts^

ts;

ts,

ts.

ts-

ts_

ts)

ts(

.ts

.ts^

.ts;

.ts,

.ts_

.ts)

.ts[

^ts

^ts^

^ts;

^ts,

^ts.

^ts)

^ts[

^ts"

t^s

-ts

-ts,

-ts.

-ts-

-ts)

-ts"

Keywords

-ts1
 -ts2
 -ts3
 _ts
 _ts;
 _ts,
 _ts.
 ts
 _ts)
 _ts(
 _ts[
 _ts"
 \ta
 \ta[
 .ta;
 .ta[
 \tm
 \tm;
 \tm,
 \tm[
 \tm1
 .tm
 .tm[

Keywords

123199
 1299
 9912
 1900
 99365
 99366
 4912
 century
 dob
 dte
 julian
 last^update
 \last-update
 last^used
 last-used
 last_used
 leap
 period
 stamp
 stmp
 timest
 timst
 tme
 tmst
 year
 ymd
 yr
 yy

Guardian

Procedures

dayofweek
 interpretinterval

BAUTILS

get^day^in^week
 omf^file^cleanup
 rttbf^purge
 rttf^purge
 week^day

ACI Utilities

bump^day
 day^of^week
 get^next^day
 test^holiday
 test^weekend

Appendix B - Tandem and ACI Date and Time Representations

This section outlines some date and time representations used by Tandem and ACI. It also contains information on possible problems or considerations to be used with specific date/time routines. Problems outlined here represent types of problems already found, not necessarily all types of problems that exist. The purpose is to provide clues on where to look for potential date/time related problems.

Definitions

- Tandem's publications refer to microseconds, milliseconds and centiseconds.
1 microsecond = $1/1,000,000$ second = .000001 second
1 millisecond = $1/1000$ second = .001 second
1 centisecond = $1/100$ second = .01 second
- In the examples that follow, the returned value for each routine represents a date and time of noon, January 1, 2000.

Date and Time Representations

GREGORIAN ARRAY

```
int dat^and^tim[0:6]  
call time (dat^and^tim);
```

When viewed in INSPECT, dat^and^tim might look like this:

```
2000 1 1 12 00 00 00  
yyyy mm dd hh mm ss hh
```

A Gregorian array is a calendar representation of a date and time. For this reason it should be evaluated carefully.

POSSIBLE COMPARISON PROBLEMS: Generally, comparisons between Gregorian arrays are safe because the "year" portion is represented in 4 digits. However, conversions of 4-digit years to 2-digit years have been known to happen, so be careful. When Gregorian arrays are compared, be sure the years are represented in 4 digits.

POSSIBLE CONVERSION PROBLEMS: Occasionally, the 4-digit year of a Gregorian Array is converted to a 2-digit year by always subtracting 1900. This conversion will yield an invalid year when years greater than 1999 are

converted. Likewise, a 2-digit year might be converted to a 4-digit year by always adding 1900.

48-BIT TIMESTAMP

```
int tstamp[0:2]  
call timestamp (tstamp);
```

When viewed in INSPECT, tstamp might look like this:

18 24343 -19405

This is a quantity equal to the number of centiseconds since 00:00 December 31, 1974. 48-bit timestamps are always LCT.

POSSIBLE COMPARISON PROBLEMS: None, since this is a quantity.

POSSIBLE CONVERSION PROBLEMS: None, as long as the Tandem date and time system procedures or the ACI Utilities are used properly.

64-BIT JULIAN TIMESTAMP

```
fixed jul^tstamp  
jul^tstamp := juliantimestamp
```

When viewed in INSPECT, jul^tstamp might look like this:

211,813,509,604,024,960

This is a quantity equal to the number of microseconds since 12:00 GMT, January 1, 4713 B.C.

POSSIBLE COMPARISON PROBLEMS: None, since this is a quantity.

POSSIBLE CONVERSION PROBLEMS: None, as long as the Tandem date and time system procedures or the ACI Utilities are used properly.

ASCII CALENDAR DATES

```
string yymmdd[0:5]
```

When viewed in INSPECT, yymmdd might look like this:

000101

POSSIBLE COMPARISON PROBLEMS: Any comparison of dates in this format should be replaced with a call to COMPARE^ASCII^YYMMDD in TAL code or COMPARE-ASCII-YYMMDD in COBOL code. These utilities will

convert YY year to a 4-digit year. These utilities determine the century by evaluating the 2-digit year. If YY is less than 75, the century portion of the date will be 20. If YY is greater than or equal to 75, the century portion of the date will be 19. More detail on the utilities can be found in Appendix C.

POSSIBLE CONVERSION PROBLEMS: Problems occur when changing a date with a 2-digit year to a date with a 4-digit year and vice versa. In some situations, the number 1900 is added or subtracted to the year. In other situations, the current system time is used to determine if 1900 or 2000 should be added or subtracted (which works only if the date being processed is in the current century).

BINARY CALENDAR DATE

`int(32) yymmdd`

When viewed in INSPECT, yymmdd might look like this (leading zeros are suppressed):

101

POSSIBLE COMPARISON PROBLEMS: Any comparison of dates in this format should be replaced with a call to COMPARE^BINARY^YYMMDD. This utility will convert YY year to a 4-digit year. The utility determines the century by evaluating the 2-digit year. If YY is less than 75, the century portion of the date will be 20. If YY is greater than or equal to 75, the century portion of the date will be 19. More information on this utility can be found in Appendix C.

POSSIBLE CONVERSION PROBLEMS: Problems occur when changing a date with a 2-digit year to a date with a 4-digit year and vice versa. In some situations, the number 1900 is added or subtracted to the year. In other situations, the current system time is used to determine if 1900 or 2000 should be added or subtracted (which works only if the date being processed is in the current century).

LEAP YEAR PROCESSING

The ACI Utility LEAP^YEAR has been tested and works correctly. The year 2000 IS a leap year. If a program is doing its own leap year determination, it should be evaluated carefully. Modules have been found which do their own leap year determination and do it incorrectly. If there is a problem, replace the program logic with a call to LEAP^YEAR.

Tandem Date and Time System Procedures

Here is a list of Tandem's system procedures that deal with dates and times.

Brief summaries are provided for selected procedures. If you want more information, refer to the Guardian Procedure Calls Reference Manual.

COMPUTEJULIANDAYNO	Converts Gregorian date to Julian day number
COMPUTETIMESTAMP	Converts Gregorian timestamp to Julian
CONTIME	Converts 48-bit timestamp to Gregorian
CONVERTTIMESTAMP	Converts GMT to/from local time
DAYOFWEEK	Converts Julian day number to day of week
INTERPRETINTERVAL	Converts microseconds to ddhhmmsshtt
INTERPRETJULIANDAYNO	Converts Julian day number to YYYY,MM,DD
INTERPRETTIMESTAMP	Converts Julian timestamp to Gregorian
JULIANTIMESTAMP	Generates Julian timestamp (64-bit)
TIME array)	Generates Gregorian timestamp (7-word
TIMESTAMP	Generates 48-bit timestamp

ACI Date and Time Utilities

Here is a list of the date and time procedures in the ACI Utilities. If you want more information, refer to the ACI Utilities Manual.

ASCII^JULIAN^TIMESTAMP	JULIAN^TIMESTAMP^ASCII
ASCII^TIMESTAMP	JULIAN^TO^YYMMDD
BUMP^DAY	LEAP^YEAR
BUMP^YYMMDD	LONG^DATE^AND^TIME
CLOCK^TIME	TEST^HOLIDAY
COMPARE^ASCII^YYMMDD	TEST^WEEKEND
COMPARE^BINARY^YYMMDD	TIMESTAMP^ADJUST^NUM^MINUTES
CONTIME	TIMESTAMP^ASCII
CONVERT^JULIAN^TO^GREG	TIME^ASCII
CONVERT^TIME^TO^YYMMDD	TIME^TIMESTAMP
CONVERT^TIMESTAMP^FORMAT	TIME^ZONE^CHANGE
CONVERT^YYMMDD^TO^TIME	TIME^ZONE^CHANGEX
CURRENT^TIME^AS^STRING	TODAYS^DATE
DAY^OF^WEEK	VALID^DAT^TIM^STRING
GET^DAY^OF^WEEK	VALID^DATE^YYMMDD
GET^NEXT^DAY	VALID^TIM^STRING
JULIAN	YYMMDD^TO^JULIAN
JULIAN^DATE	YY^DDD^TO^TIME

Appendix C - Standard Solutions to Common Year 2000 Problems

This section describes common problems that have been observed during Year 2000 analysis and testing of BASE24 Release 5.1/1.1. For each common problem, a standard solution is suggested. For the sake of consistency and maintainability across our product line, these standard solutions should be implemented wherever possible.

NOTE: Examples are provided to illustrate concepts; they should not be taken literally. Developers should keep in mind that there are many variations of the problems and solutions outlined in this document.

Determination of Century

PROBLEM: When presented with a date/time in which the year is represented by only 2 digits (e.g., a string variable in the format YYMMDDHHMMSSHH), some programs and utilities may have to determine the century and convert to a 4 digit year (YYYY). One programming practice that has been observed gets the current date/time via a call to TIME (or TIMESTAMP, etc.), then derives the century from the current date/time.

```
string yymmddhhmmsshh[ 0:13 ] := "99123112000000";
int    time^array[ 0:6 ];
int    century;
int    year;

call time( time^array );
century := time^array[ 0 ] / 100 * 100;
                                ! century = 1900
call ascii^integer( yymmddhhmmsshh[ 0 ], 2, year );
                                ! year = 99
year := year + century;         ! 1999 = 99 + 1900
```

The example above derives a century of 1900 if time^array[0] is in the range 1900-1999, and 2000 if time^array[0] is in the range 2000-2099. Problems arise when the date/time to be processed is not in the current century. Using the example above, if YYMMDDHHMMSSHH = 00010112000000 (12:00 PM on January 1, 2000) and the current, actual year is 1999, YEAR will be erroneously calculated as 1900.

SOLUTION: Determine the century by making assumptions. If a 2 digit year falls in the range 00-74, assume the century is 2000. If a 2 digit year falls in

the range 75-99, assume the century is 1900. For example:

```
call ascii^integer( yymmddhhmmssshh[ 0 ], 2, year );
if year < 75 then
    century := 2000
else
    century := 1900;
year := year + century;
```

Comparing YYMMDD Dates

PROBLEM: Dates are often compared to determine if one is greater than or less than another. Sometimes binary values are compared, other times ASCII arrays are compared. Problems arise when 20th century dates in YYMMDD format are compared with 21st century dates in the same format. For instance, the binary representation of December 31, 1999 is 991231 (an INT(32) value) and the binary representation of January 1, 2000 is 101 (000101). As a calendar date, December 31, 1999 is less than January 1, 2000. But as a binary value, 991231 is greater than 101. Likewise, the ASCII representation of December 31, 1999 is "991231" (a STRING array) and the ASCII representation of January 1, 2000 is "000101". Again, as a calendar date, December 31, 1999 is less than January 1, 2000. But as an ASCII array, "991231" is greater than "000101".

SOLUTION: If two dates in YYMMDD format are compared using the relational operators <, >, <=, >=, '<', '>', '<=' or '>=', replace the comparison with a call to one of these three new ACI Utility procedures. The TAL utilities are in BA10UTIL.UTILS and the COBOL utility is in BA51SRC.BACOUTILS.

COMPARE^ASCII^YYMMDD for TAL code

This procedure compares, according to the supplied operator, two 6-byte string dates in YYMMDD format. Prior to comparison, the 6-byte dates are expanded from YYMMDD to YYYYMMDD format, allowing comparisons between dates in the 20th and 21st centuries. If a supplied date's 2-digit year YY falls in the range 00-74, the date is expanded to 20YYMMDD. If a supplied date's 2-digit year YY falls in the range 75-99, the date is expanded to 19YYMMDD. NOTE: If either supplied date is all spaces, zeroes or binary zeroes, it is expanded to eight bytes of spaces, zeroes or binary zeroes, respectively. The syntax for this procedure call is as follows:

```
return^val := COMPARE^ASCII^YYMMDD( yymmdd^1      ! i
                                     ,operator      ! i
```

,ymmdd^2); ! i

return^val INT returned value

Returns TRUE if the comparison evaluates to TRUE; returns FALSE if the comparison evaluates to FALSE.

ymmdd^1 STRING:ref:6 input

The left half of the comparison - a 6-byte ASCII date in YYMMDD format.

operator INT:value input

The relational operator indicating how to compare the halves. Following are the possible values.

<u>Value</u>	<u>Operator</u>	<u>Description</u>
1	<	signed less than
2	>	signed greater than
3	<=	signed less than or equal to
4	>=	signed greater than or equal to
5	'<'	unsigned less than
6	'>'	unsigned greater than
7	'<='	unsigned less than or equal to
8	'>='	unsigned greater than or equal to

ymmdd^2 STRING:ref:6 input

The right half of the comparison - a 6-byte ASCII date in YYMMDD format.

COMPARE^BINARY^YYMMDD

This procedure compares, according to the supplied operator, two binary dates in YYMMDD format. Prior to comparison, the binary dates are expanded from YYMMDD to YYYYMMDD format, allowing comparisons between dates in the 20th and 21st centuries. If a supplied date's 2-digit year YY falls in the range 00-74, the date is expanded to 20YYMMDD. If a supplied date's 2-digit year YY falls in the range 75-99, the date is expanded to 19YYMMDD. NOTE: If either supplied date is zero, it remains zero for the comparison. The syntax for this procedure call is as follows:

return^val := COMPARE^BINARY^YYMMDD(*dat^1* ! i
,operator ! i
,dat^2); ! i

return^val INT returned value

Returns TRUE if the comparison evaluates to TRUE; returns FALSE if the comparison evaluates to FALSE.

dat^1 INT(32):value input

The left half of the comparison - a binary date in YYMMDD format.

operator INT:value input

The relational operator indicating how to compare the halves. Following are the possible values.

<u>Value</u>	<u>Operator</u>	<u>Description</u>
1	<	signed less than
2	>	signed greater than
3	<=	signed less than or equal to
4	>=	signed greater than or equal to

dat^2 INT(32):value input

The right half of the comparison - a binary date in YYMMDD format.

COMPARE-ASCII-YYMMDD for COBOL code

This procedure compares, according to the supplied operator, two 6-byte string dates in YYMMDD format. Prior to comparison, the 6-byte dates are expanded from YYMMDD to YYYYMMDD format, allowing comparisons between dates in the 20th and 21st centuries. If a supplied date's 2-digit year YY falls in the range 00-74, the date is expanded to 20YYMMDD. If a supplied date's 2-digit year YY falls in the range 75-99, the date is expanded to 19YYMMDD. NOTE: If either supplied date is all spaces or zeroes, it is expanded to eight bytes of spaces or zeroes, respectively. The syntax for this COBOL utility call is as follows:

PERFORM 480-COMPARE-ASCII-YYMMDD

Prior to performing this utility, several fields used by the utility need to be initialized: *ws-yymmdd-1*, *ws-yymmdd-2* and *ws-operator*.

ws-yymmdd-1 STRING:6

The left half of the comparison - a 6-byte ASCII date in YYMMDD format.

ws-yyymmdd-2 STRING:6

The right half of the comparison - a 6-byte ASCII date in YYMMDD format.

ws-operator STRING:1

The relational operator indicating how to compare the halves. Following are the possible values.

Value	Operator	Description
1	<	less than
2	>	greater than
3	<=	less than or equal to
4	>=	greater than or equal to

The utility sets a working storage variable, WS-COMPARE-FLG, to “Y” if the comparison passed the check, otherwise, it sets the flag to “N”. The calling module then can check a condition-name, *ws-compare-pass* or *ws-compare-fail*, to determine if the date check was passed or failed.

Example:

If an existing date check is as follows:

```
IF ( DAT OF PIRF < YYMMDD-START )
```

```
...
```

it would be replaced by:

```
MOVE DAT OF PIRF TO WS-YYMMDD-1
```

```
MOVE YYMMDD-START TO WS-YYMMDD-2
```

```
MOVE “1” TO WS-OPERATOR
```

```
PERFORM 480-COMPARE-ASCII-YYMMDD
```

```
IF WS-COMPARE-PASS
```

```
... (processing to be performed if DAT of PIRF < YYMMDD-  
START)
```

```
ELSE
```

```
... (processing to be performed if DAT of PIRF is not <  
YYMMDD-START)
```

Conversion of a Four-Digit Year to a Two-Digit Year

PROBLEM: Some programs and utilities convert a 4-digit year to a 2-digit

year by subtracting 1900 from the 4-digit year. If the 4-digit year is greater than 1999, the result is invalid. For example, suppose January 1, 2000 12:00:00 is represented by the following 7-word Tandem time array:

2000 1 1 12 0 0 0

If century 1900 is always assumed, then $2000 - 1900 = 100$, which is not a valid year.

SOLUTION: If a 4-digit year is less than 2000, subtract 1900 to obtain a 2-digit year. Else, subtract 2000.

Conversion of a Two-Digit Year to a Four-Digit Year

PROBLEM: Some programs and utilities convert a 2-digit year to a 4-digit year by adding 1900 to the 2-digit year. If the 2-digit year is in the range 00-74, the result is invalid. For example, suppose an expiration date of January, 2000 is represented in YYMM format as:

0001

If century 1900 is always assumed, then $00 + 1900 = 1900$, which is not the correct year for the expiration date.

SOLUTION: If a 2-digit year is in the range 00-74, add 2000 to obtain a 4-digit year. If a 2-digit year is in the range 75-99, add 1900.

Application File Names in YYMMDD Format

PROBLEM: The filenames of a number of BASE24 application files are in YYMMDD format. For instance, the filename format of the POS Transaction Log File (PTLF) is POYYMMDD, where YYMMDD = the date on which transactions are posted to the log file. Date comparisons between two such files occasionally go unreported by SEEK. Consider the following example:

curr^ptlf is an array containing the string "\$VOL SUBVOL
PO991231"

nxt^ptlf is an array containing the string "\$VOL SUBVOL
PO000101"

```
if curr^ptlf[ 18 ] <= nxt^ptlf[ 18 ] for 6 then
  begin
    !
```

```
! do something  
!  
end;
```

Because the arrays are compared directly and the array names (curr^ptlf and nxt^ptlf) are not in the Year 2000 Master Keyword List, SEEK could not output this problematic comparison to its Master Out File. If the developer relied on the analysis of SEEK's Master Out File only, and not on a more in-depth analysis of the module's entire source file(s), this comparison could go undetected unless encountered during testing.

SOLUTION: Be particularly careful when analyzing BASE24 modules that process YYMMDD-format filenames. Such files include the TLF, PTLF, TTLF, ILF and report obey files, among others. In addition to the usual analysis of SEEK's Master Out File, you should search the module's source file(s) for all occurrences of "<" and ">" and look for comparisons similar to the example above. This extra analysis may seem tedious, but in reality it is relatively easy to do and will give you an added measure of confidence that all Year 2000 problems have been found.

Date Fields Within Primary or Alternate Keys

PROBLEM: A number of BASE24 data files have primary or alternate keys that contain date fields in YYMMDD or YYMM format. Read operations performed on these files may not produce the expected results - some records may be read out of order or skipped.

For example, suppose the primary key of a key-sequenced file contains a date field in YYMMDD format. The file contains three records logically stored in order of ascending primary key values: 000101, 991230 and 991231. A module beginning a sequential read operation at the beginning of the file will read the records in this order: 000101, 991230, 991231. From the perspective of ascending calendar dates, these records appear out of order. A module beginning a sequential read operation at record 991230 will read records 991230 and 991231, but 000101 will be skipped.

SOLUTION: The potential problems are not as serious as they appear. In keeping with ACI's goal of minimal system impacts, it is suggested that no changes be made to the formats of these files, nor should changes be made to the manner in which BASE24 modules process the files.

Those modules that encounter "exceptions" are documented in the Release 5.1/1.1 Sync Point 02 Implementation Guide. Included in the documentation are: (a) a description of a particular operation performed by the module and the expected outcome under normal circumstances, (b) a description of the

exception, or deviation from the expected outcome, and the circumstances that prompted the exception, (c) an estimate of the exception's duration and (d) a description of an operational workaround, or alternative, if one is necessary.

The following BASE24 data files have been identified in Release 5.1/1.1 as containing date fields within primary or alternate keys and for which exceptions have been found:

File	Product
POS Transaction Log File (PTLF)	POS
POS Institution Reporting File (PIRF)	POS
No Book File (NBF)	Teller
Warning/Hold/Float File (WHFF)	Teller
Teller Transaction Log File (TTLF)	Teller
Report Transactions by Time Block File (RTTB)	Teller
Report Transaction by Type File (RTTF)	Teller
Banknet Request Log File (BNRLF)	Interchange Interfaces
PLUS File Maintenance Log File (PMLF)	Interchange Interfaces
Visanet File Request Log File (FRLF)	Interchange Interfaces
Update Log File (ULF)	From Host Maintenance
Host Mail Box File (HMBF)	Mail
Mail Box File (MBF)	Mail
Fraud Parameters File (FPF)	Customer Service/Fraud Control
Fraud Suspect File (FSF)	Customer Service/Fraud Control

The following BASE24 data files have been identified in Release 5.1/1.1 as containing date fields within primary or alternate keys and for which no exceptions have been found:

File	Product
POS Retailer Reporting File (PRRF)	POS
Institution Periodic Reporting File (IPRF)	Core
Monthly Transaction Accumulation File (MTAF)	Transaction Based Pricing

Look for other files of this type. Carefully evaluate modules that read files of this type. Identify acceptable workarounds.

Card Expiration Dates

PROBLEM: BASE24 sometimes finds it necessary to assign a default expiration date when processing a transaction, such as when track 2 contains no expiration date or when building a manual track 2. A default expiration date of "9912" is no longer acceptable. Other unacceptable defaults that

have been found are “4912” and “4999”.

SOLUTION: Make all card expiration dates have a standard default of “0000”.

NOTE: Historically, BASE24 has accepted expiration dates in YYMM or MMYYY format from external entities and has processed those expiration dates internally in YYMM format. BASE24 will continue to handle card expiration dates in the same way with one notable assumption. Because it is impossible to distinguish between the month and year portions of expiration dates that fall between January, 2001 (0101) and December 2012 (1212), it must be assumed that expiration dates in this range are in YYMM format.

ACI Worldwide Inc.